HRMS Dashboard Redesign Guide

A practical overview of the role-aware workspace, employee self-service, technology stack, and local setup.

One workspace for people operations
Role-aware navigation keeps each task in its own page while the dashboard summarizes the most useful live information.

Role-based workspace

Admin manages people, departments, attendance, leave, payroll, reports, roles, audit history, and organization settings. **HR** has the assigned people operations, payroll, and reporting functions. **Employee** can view their own profile, attendance, time off, and salary slips.

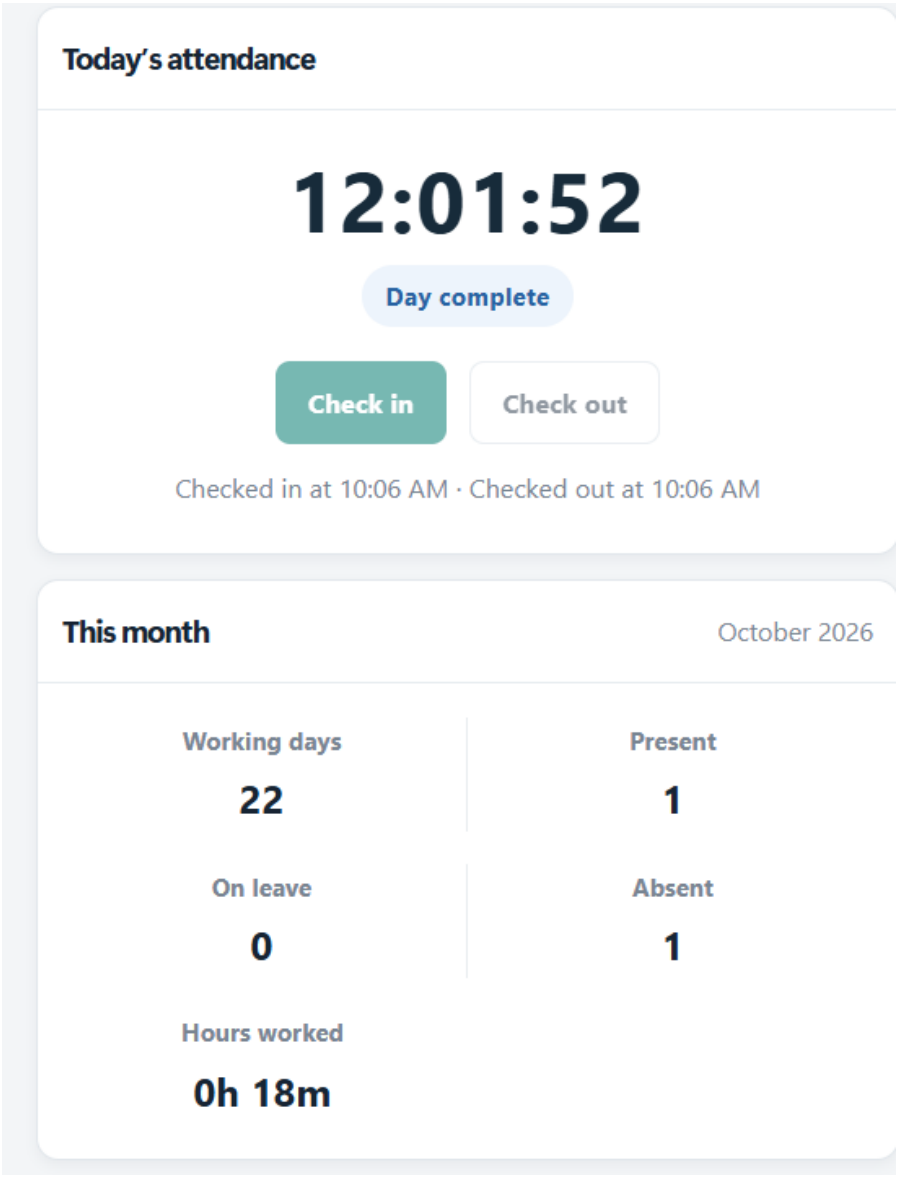
Technology

React 18, React Router, Vite, Bootstrap, Axios, FastAPI, SQLAlchemy, Pydantic Settings, JWT authentication, SQLite for local development, and MySQL 8+ through the PyMySQL SQLAlchemy driver.

The screenshots in this guide are genuine captures of the authenticated Employee workspace. Admin/HR-only screens are described below but are not shown because an authorized Admin/HR browser session was not available.

Dashboard at a glance

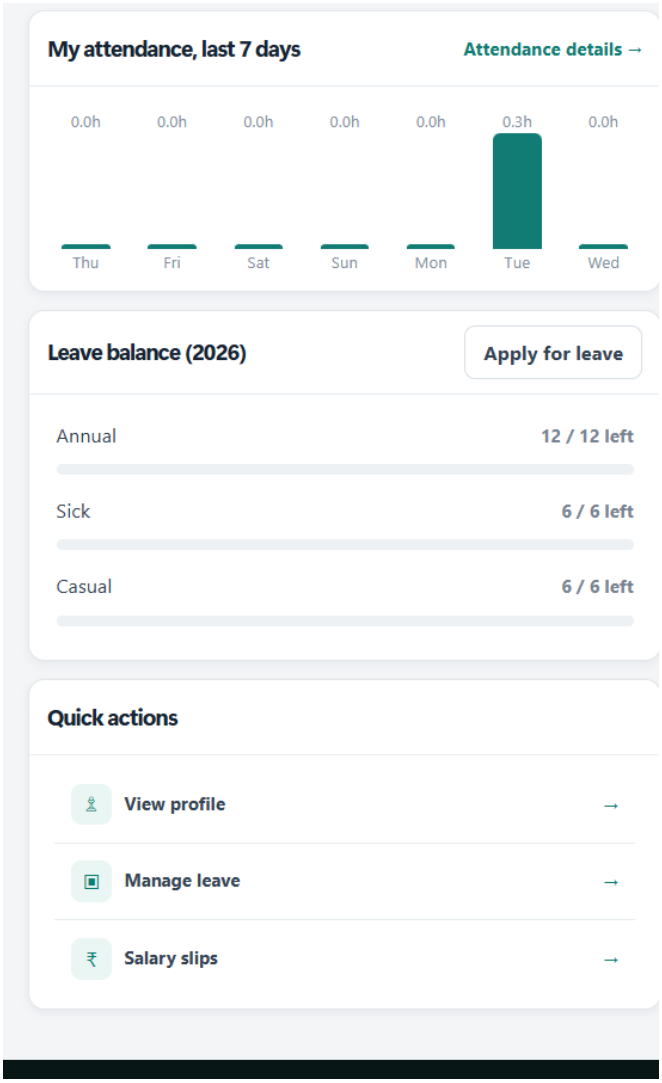
The dashboard brings daily attendance, the monthly snapshot, a seven-day attendance chart, leave balances, and quick links into one responsive page. Values are loaded from the authenticated API rather than fixed dashboard copy.



Screenshot excerpt: employee dashboard header, attendance status, and monthly summary.

Insights and separate modules

The dashboard's lower section keeps the attendance chart, leave balance, and quick actions together without embedding the full management modules in the overview.



Dedicated pages: Employee Management supports directory search, department filtering, pagination, add/edit forms, and detailed work/contact/compensation profiles with optional date of birth, address, and emergency-contact fields; Attendance provides month/year filtering and records; Leave shows balances and request history; Payroll presents personal salary-slip history or authorized team payroll. Departments, Reports, Notifications, User Management, Audit Logs, Settings, and My Profile each have separate routes.

HR/Admin page access is additionally enforced by the FastAPI API permissions. Frontend role-based navigation and route guards are a usability layer, not the security boundary.

Run locally

1. Backend (PowerShell)

Run from the project root. Configure a unique JWT secret and admin password locally; never use the example placeholders as production credentials.

```
cd backend
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
Copy-Item .env.example .env
# Edit backend/.env: set a unique JWT_SECRET_KEY (32+ characters)
$env:HRMS_ADMIN_EMAIL="admin@example.com"
$env:HRMS_ADMIN_PASSWORD="<your-unique-password>"
python -m app.seed
uvicorn app.main:app --reload
```

2. Frontend (second PowerShell terminal)

```
cd frontend
npm install
npm run dev
```

Open http://localhost:5173. The API defaults to http://localhost:8000/api. For MySQL 8+, create the database/user with database/schema-notes.sql and set DATABASE_URL in backend/.env to a mysql+pymysql:// connection URL. URL-encode reserved password characters. SQLite remains the default for a quick local start.

Validation

From frontend, run npm run build. From backend, run .\run_live_checks.ps1 in PowerShell for the repository's isolated live API checks and workflow tests.

More detail: see README.md, backend/.env.example, frontend/.env.example, and database/schema-notes.sql.